Post-Quantum Retrofit on StarFive VisionFive 2: Results Summary

Date: 2026-11-10. Device: StarFive VisionFive 2 (JH7110, 4x SiFive U74 RV64GC @ 1.5 GHz, no vector unit), Debian.

1. Result in one sentence

On the VisionFive 2, a TLS 1.3 connection with **hybrid key establishment (X25519 + ML-KEM-768) and hybrid authentication (ECDSA P-256 + ML-DSA-44)** completes in **10.2–10.3 ms end-to-end**, versus **10.7 ms for the baseline (X25519 + RSA-3072)**. That means **no increase in end-to-end latency**, on the same device, network and server.

2. What was built

Part	What it is
OpenSSL 3.5.9 (`/opt/openssl-3.5`)	Built from source on the board; native ML-KEM and ML-DSA. The system OpenSSL 3.0.9 is untouched.
OpenSSL 3.5.9 "opt" (`~/openssl-3.5-opt`)	Same, with `enable-ec_nistp_64_gcc_128` (faster, constant-time P-256).
`hybrid_server` (Mac)	TLS 1.3 server holding an RSA-3072 certificate (legacy clients) and an ECDSA P-256 certificate plus ML-DSA-44/65 keys (hybrid clients).
`hybrid_client` (board)	Benchmark client. Really authenticates the server (pinned certificate, pinned ML-DSA key). Measures latency, bytes and memory.
`pqc_client` (board)	Key-exchange-only client used against the hackathon server.

How hybrid authentication works (dual signature, bound to the session)

1. Normal TLS 1.3 handshake, server signs with **ECDSA P-256** (classical).
2. Server also signs `"hybrid-auth-v1" || TLS-exporter(32 bytes)` with **ML-DSA** (post-quantum). The exporter value is unique to this connection's handshake, so the signature cannot be replayed elsewhere.
3. The ML-DSA signature travels in the first HTTP response, so there's **no extra round trip**.
4. The client accepts only if **both** signatures verify, so an attacker must break both ECDSA and ML-DSA.
5. Legacy RSA clients keep working against the same server (backward compatible).

Security checks done: a wrong ML-DSA public key is rejected; a wrong pinned certificate is rejected.

3. Benchmark results

Board → Mac over local Ethernet (~1 ms RTT, so crypto dominates; the worst case for PQC). CPU governor `performance`. 50 connections per configuration, run forward and reverse; values are the mean of the two pass medians.

#	Configuration	End-to-end	vs baseline	TLS handshake	PQ verify	Bytes rx / tx
A	**Baseline:** X25519 + RSA-3072	**10.66 ms**	-	8.59 ms	-	2,520 / 330
B	Hybrid v1: X25519MLKEM768 + ECDSA + ML-DSA-65	14.24 ms	+3.58 ms	8.54 ms	1.69 ms	7,095 / 1,526
C	B + fast P-256 build	11.86 ms	+1.20 ms	5.71 ms	1.69 ms	7,095 / 1,526
D	C + ML-DSA-44 instead of 65	**10.24 ms**	**–0.42 ms**	5.80 ms	1.09 ms	5,911 / 1,526
E	D + binary signature (no base64)	**10.32 ms**	**–0.34 ms**	5.91 ms	1.06 ms	5,105 / 1,552
H	Baseline built with the "opt" library (fairness check)	10.91 ms	+0.25 ms	8.84 ms	-	2,520 / 330

Per-pass end-to-end medians: baseline 10.33 / 11.00 ms (A) and 10.49 / 11.33 ms (H); final hybrid 10.29 / 10.19 ms (D) and 10.05 / 10.59 ms (E). The difference is within measurement noise, so the honest claim is **"no increase"**, not "faster".

Session resumption (repeat connections)

Configuration	End-to-end (resumed)
G: Baseline X25519 + RSA, resumed	4.60 ms
F: Hybrid (E) resumed	5.98 ms

A resumed session skips all signatures (it inherits the authentication of the full hybrid handshake) but still does a fresh X25519MLKEM768 key exchange. Resumed hybrid is +1.4 ms slower than resumed baseline because of the larger ML-KEM messages, but both are about 2× faster than any full handshake.

Memory

Peak process memory (max RSS) is 6.3–6.6 MB in every configuration; hybrid adds about +0.1–0.2 MB.

Communication overhead (final hybrid E vs baseline A)

- Received: 5,105 vs 2,520 bytes (+2.6 KB): the ML-DSA-44 signature (2,420 B) plus the ML-KEM ciphertext (1,088 B), minus the smaller ECDSA certificate and signature.
- Sent: 1,552 vs 330 bytes (+1.2 KB): the ML-KEM-768 public key in the ClientHello.

4. Where the time goes (bottleneck analysis)

Signature cost on the board (OpenSSL 3.5.9, 1.5 GHz):

Operation	Standard build	Opt build	Signature size
RSA-3072 sign / verify	34.1 / 0.65 ms	34.1 / 0.64 ms	384 B
ECDSA P-256 sign / verify	5.72 / **4.11 ms**	0.64 / **1.41 ms**	71 B
ML-DSA-44 sign / verify	4.69 / 0.99 ms	same	2,420 B
ML-DSA-65 sign / verify	7.65 / 1.58 ms	same	3,309 B
X25519MLKEM768 keygen / decaps	1.43 / 1.61 ms	same	

Findings: 1. **The board-side bottleneck was ECDSA P-256, not the post-quantum part.** The default OpenSSL build uses slow generic code on RISC-V. One build flag (`enable-ec_nistp_64_gcc_128`) made verification 2.9× faster. 2. **RSA moves the cost to the server.** A handshake trace showed the server needing 3–10 ms to sign with RSA-3072, while the board verifies in 0.65 ms. Hybrid is the reverse: cheap to sign, more work to verify. 3. **If**

the device itself signs (device authentication, mutual TLS), hybrid wins clearly: ECDSA + ML-DSA-44 signing is 0.64 + 4.69 = 5.3 ms versus 34.1 ms for RSA-3072. 4. **CPU frequency matters as much as the algorithms.** With the default `ondemand` governor the CPU idles at 375 MHz; switching to `performance` cut hybrid handshakes against the real hackathon server from 62 ms to 45 ms (median, confirmed in both test orders). This is now set permanently through `cpu-performance.service`.

5. Hybrid key exchange against the real hackathon server

Server `[2a0f:7cc7:7cc7:7c40::94ad]:443` (RSA-3072, only RSA signatures accepted), `pqc_client`, 50 runs each:

CPU governor	X25519 (legacy) median	X25519MLKEM768 (hybrid-pqc) median
ondemand	57.3 / 56.9 ms	62.2 / 61.1 ms
performance	41.7 / 44.0 ms	45.3 / 45.0 ms

Server-side confirmation: `"handshake_method":"hybrid-pqc"`. Over the internet the network round trip (~40–50 ms) dominates, and hybrid key exchange adds about +1–5 ms.

6. Acceleration (Hailo-8L): assessment, not measured

- All crypto on the board's critical path is now about 2.5 ms (ECDSA 1.4 + ML-DSA-44 1.0) plus about 3 ms for ML-KEM; NTT and polynomial arithmetic are only part of that. Even infinitely fast offload saves under ~2–3 ms per handshake.
- The Hailo-8L is an int8 neural-network accelerator behind PCIe. ML-KEM works mod 3329 (12-bit) and ML-DSA mod 8380417 (23-bit), so the arithmetic would need splitting into several 8-bit limbs plus extra reduction steps. Per-call PCIe transfer and setup overhead is likely in the same range as the whole ML-DSA verification.
- Conclusion: offloading is unlikely to pay off for single handshakes. The measured software fixes (P-256 build flag, ML-DSA-44, CPU governor) gave more than an accelerator could.
- **Not verified on hardware**; a microbenchmark of Hailo transfer latency would confirm or refute this.

7. Migration guide: lessons learned

1. **Upgrade the TLS library first.** Debian images for this board ship OpenSSL 3.0.9 with no ML-KEM or ML-DSA; OpenSSL 3.5 LTS has both built in. Install it to a separate prefix with an rpath; don't replace the system libssl. Building takes about 30–40 minutes on the board.
2. **Check the server.** Hybrid authentication needs server support; the hackathon server only offers RSA-3072.
3. **Check the CPU governor.** `ondemand` leaves short TLS bursts running at 375 MHz.
4. **Build OpenSSL with `enable-ec_nistp_64_gcc_128` on 64-bit RISC-V.** It makes P-256 several times faster.
5. **Pick matching security levels.** P-256 + ML-DSA-44 (the pairing used in the IETF composite-signature draft) is enough and faster/smaller than ML-DSA-65.
6. **Send signatures as raw bytes, not base64** (saves ~800 B).
7. **Use session resumption** for devices that reconnect often: no signatures, fresh PQ key exchange.
8. **Pitfalls hit along the way:** `dhclient -r` broke IPv6 under NetworkManager (fix: `nmcli con up`); a local TCP relay inflated the "legacy" timing to 165 ms by counting the relay's own connect time; Python's `ssl` module is tied to the system OpenSSL and cannot use the hybrid groups.

8. Limitations (be upfront about these)

- The hybrid-authentication server is our own (on the Mac), not the hackathon server, which cannot do PQ signatures. Baseline and hybrid were measured against the same server for fairness.
- Local network only (~1 ms RTT) for the authentication benchmarks; over the internet both would include ~40–50 ms RTT.
- The dual-signature scheme is a custom construction, not a standardised TLS signature algorithm; certificates and the ML-DSA key are pinned, not issued by a PKI. Standards-track options: composite ML-DSA certificates (IETF LAMPS draft) or X.509 alternative-key ("Catalyst") certificates.
- Energy was not measured (it needs a USB-C power meter on the board's supply).
- The Hailo-8L analysis is an estimate, not a measurement.
- "No increase" is within noise (about ±0.5 ms between passes); hybrid isn't claimed to be faster.

9. How to reproduce

On the Mac: `cd ~/pqc && ./hybrid_server 8444 keys` On the board (`cd ~/pqc`):

```
./hybrid_client_std 192.168.2.1 8444 X25519 rsa 50                                  # baseline
./hybrid_client_opt 192.168.2.1 8444 X25519MLKEM768 hybrid44 50 keys binary          # final
hybrid
./hybrid_client_opt 192.168.2.1 8444 X25519MLKEM768 hybrid44 50 keys binary resume   # with
resumption
./bench_final.sh 50                                                                  # full
table -> final_results.csv
TRACE=1 ./hybrid_client_opt 192.168.2.1 8444 X25519MLKEM768 hybrid44 1 keys binary  # per-
message timeline
./sigbench 100 ; ./sigbench_opt 100                                                  #
signature micro-benchmarks
./pqc_client X25519MLKEM768 10                                                       #
hackathon server, hybrid KEX
```

Raw data on the board: `~/pqc/final_results.csv`, `ab_results.csv`, `ab_results_rev.csv`, `auth_results.csv`.

10. BASELINE (2026-11-11): default CPU clock, 1000 runs per mode

The CPU governor was switched back to `ondemand` (default) because the chip was warming up. Rerun with 1000 measurements per mode, alternating blocks (full report: `baseline_default_clock/baseline_REPORT_default_clock.md`):

Scenario	Legacy median (best / worst)	Hybrid median (best / worst)	Difference
Hackathon server, IPv4, handshake	39.92 ms (31.61 / 524.45)	45.08 ms (36.33 / 359.32)	+5.16 ms (+12.9 %)
Own server, LAN, end-to-end	21.61 ms (14.77 / 31.30)	27.95 ms (20.51 / 40.05)	+6.35 ms (+29.4 %)

Conclusion: the "no increase in latency" result in section 3 holds only with the CPU at full speed (`performance`). At the default `ondemand` clock, hybrid PQC is consistently slower (19/20 and 20/20 blocks). Low-latency hybrid PQC on this board therefore depends on the clock policy as well as on the algorithms; that's a heat/energy vs latency trade-off.

11. DYNAMIC CPU FREQ (2026-11-11): raise only the minimum CPU clock

Idea: keep the power-saving `ondemand` governor but raise `scaling_min_freq` from 312.5 MHz to 750 MHz, so a handshake no longer starts at the lowest clock. Screening of 4 settings, then a confirmation run alternating the default (D0) and min 750 MHz (D1) block by block, 1000 runs per mode (full reports: `dynamic_cpu_freq/dynamic_cpu_freq_screen_REPORT.md`, `dynamic_cpu_freq/dynamic_cpu_freq_REPORT.md`).

Scenario	Baseline: legacy, default clock	Hybrid, default clock	Hybrid, min 750 MHz
Hackathon server, handshake (median)	42.17 ms	48.30 ms	**40.26 ms** (–1.9 ms vs baseline)
Own server, full hybrid auth, end-to-end (median)	21.30 ms	27.84 ms	**16.19 ms** (–5.1 ms vs baseline)
Idle temperature	44.7 °C	44.7 °C	**45.2 °C** (+0.5 °C)

Conclusion: raising only the minimum CPU clock meets the "no increase in latency" target at the default power-saving governor, for +0.5 °C instead of the ~+6 °C of full speed. Under identical CPU settings hybrid remains slightly slower than legacy (+0.5 ms LAN, +3–4.5 ms internet).

Recommended CPU setting: `ondemand` with `scaling_min_freq` = 750 MHz. Comparison of all tested settings (own server, full hybrid authentication, end-to-end median):

Setting	Legacy	Hybrid	Idle temp	Verdict
Default (ondemand, min 312.5 MHz)	21.30 ms	27.84 ms	44.7 °C	❌ hybrid misses the target
ondemand, min 750 MHz	**15.70 ms**	**16.19 ms**	**45.2 °C**	✅ **recommended**
ondemand, min 1250 MHz	17.54 ms	16.67 ms	46.6 °C	✅ same speed, warmer
schedutil, min 312.5 MHz	23.90 ms	33.85 ms	46.2 °C	❌ worse than default
schedutil, min 750 MHz	17.58 ms	16.54 ms	46.6 °C	✅ same speed, warmer
performance (fixed 1.5 GHz, 2026-11-10)	10.66 ms	10.32 ms	~50–52 °C	✅ fastest, hottest

Full reasoning and caveats: `dynamic_cpu_freq/dynamic_cpu_freq_REPORT.md`, section "RECOMMENDATION". Not tested: 500/625 MHz minimums, energy use.

12. BITMANIP (2026-11-11): RISC-V Zba/Zbb bit-manipulation extensions

OpenSSL rebuilt with `-march=rv64gc_zba_zbb` (no code changes) so GCC uses the U74's single-instruction rotations (`rol`/`ror`) and shifted adds (`sh3add`). Full report: `bitmanip_zba_zbb/bitmanip_REPORT.md`.

Crypto at a fixed 1.5 GHz (plain → bitmanip): SHA3-256 / SHAKE-128 **+50 %**, SHA-256 +41 %, ML-KEM-768 keygen/encaps/decaps **–19 / –13 / –14 %**, ML-DSA-44/65 verify **–15 / –16 %**; RSA, ECDSA and X25519 unchanged.

End-to-end, 1000 runs per mode, default CPU clock, builds alternated block by block:

Scenario	Plain hybrid	Bitmanip hybrid	Change
Own server, full hybrid auth (median)	28.15 ms	**26.57 ms**	**–1.58 ms (–5.6 %), faster in 20/20 blocks**
Hackathon server, handshake (median)	46.69 ms	47.46 ms	no measurable difference (network noise)

Conclusion: the extensions speed up the post-quantum parts (Keccak-heavy) by 13–50 % at zero runtime cost, but end-to-end it's –1.6 ms, about 7x less than raising the minimum CPU clock (–11.6 ms). On its own it doesn't meet the latency target at the default clock. The combination (bitmanip + min 750 MHz) is the logical next step. Caveat: the library then requires a CPU with Zba/Zbb.

Source: SUMMARY.md